

PaqRap

Arquitectura de la solución integrada: Planificador + Visualizador

Entregable sol.integrada.sem08 · Proyecto H-0983 · Diseño y Desarrollo de Software 2026-2 · Grupo EQ8H · Octubre de 2026

1. Alcance de este corte

Este documento describe cómo se conectan el front (Visualizador), el back (Planificador) y la comunicación entre ambos (REST y WebSocket con STOMP) en la solución integrada que se despliega para esta entrega. Cualquier dispositivo puede conectarse por la web y ver cualquiera de los 3 escenarios: Operación Diaria, Simulación 5D y Colapso Logístico.

Incluido en este corte	Fuera de este corte
Planificador en Java con los 3 escenarios sobre el mismo motor Ingesta del archivo de pedidos subido desde la GUI Algoritmo ALNS enchufado por Strategy (ATS como stub) Visualizador en React: mapa en vivo, panel de unidad, resumen final Registro simple de pedidos en Operación Diaria Comunicación REST + WebSocket (STOMP) Despliegue web accesible desde cualquier dispositivo	Averías y bloqueos en la simulación del periodo Importación masiva distinta a la subida del archivo de pedidos Dashboards Controles de velocidad en el visualizador (RF-068): la velocidad es parámetro del backend Módulo completo de Gestión de Pedidos Algoritmo ATS funcional (queda como stub)

2. Visión general

La solución es un monolito modular: un solo backend Spring Boot (un JAR) y una aplicación web de una sola página (SPA) en React. No hay microservicios ni red entre módulos internos. El estado de cada corrida vive en memoria, sin base de datos, porque una simulación de 5 días en 30-60 minutos no debe depender de operaciones de E/S.

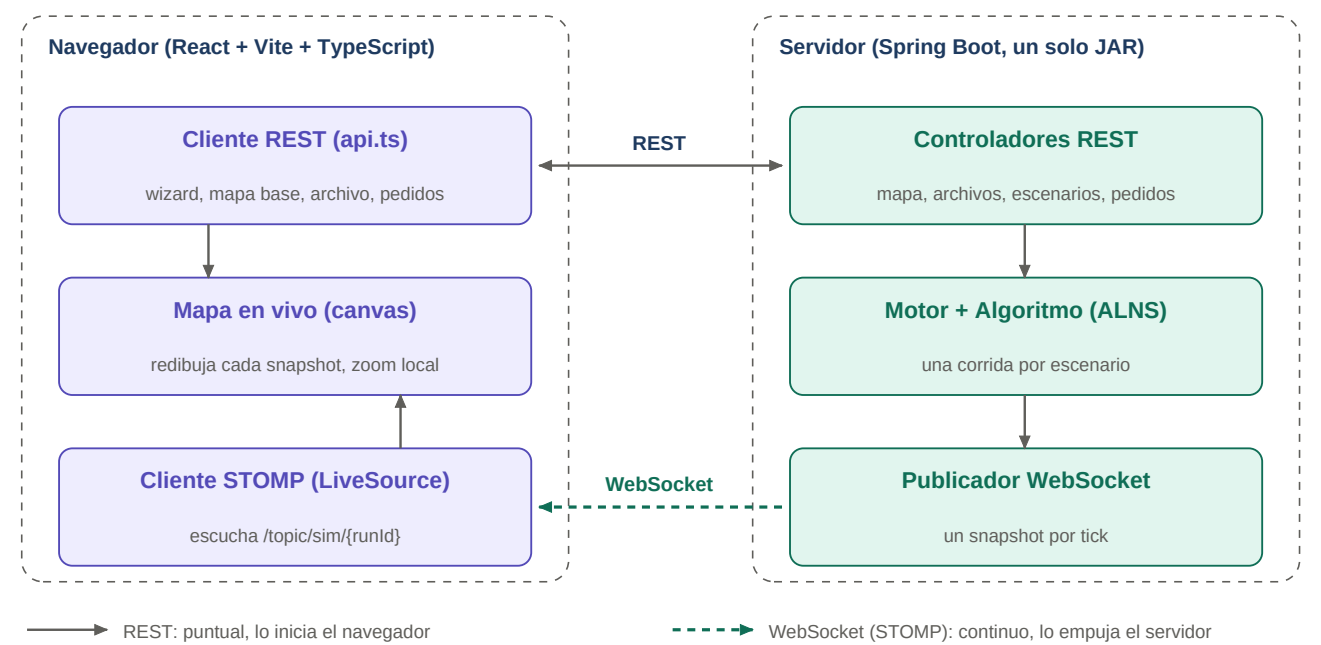


Figura 1. Arquitectura general y canales de comunicación.

Principios de diseño

- El servidor es el dueño del estado.** El front es solo un espectador: dibuja lo que recibe y no calcula rutas ni controla la velocidad.
- REST para lo puntual, WebSocket para lo continuo.** Comandos y consultas van por REST; el movimiento del mapa llega empujado por WebSocket.

- **Un solo contrato JSON** entre back y front (sección 6). Mientras no cambie, ambos lados avanzan en paralelo, y el front funciona con datos simulados (mock) hasta que el back esté listo.
- **El mismo motor para los 3 escenarios.** Solo cambian la fuente de pedidos, la velocidad del reloj y la condición de parada (ScenarioConfig).
- **Una corrida por tipo de escenario.** DIARIO, 5D y COLAPSO pueden coexistir, cada una con su reloj, motor y topic WebSocket.

3. Frontend (Visualizador)

Aplicación React + TypeScript creada con Vite, con Tailwind para estilos, react-router-dom para las rutas y @stomp/stompjs para el WebSocket. No usa Supabase, autenticación ni base de datos.

3.1 Pantallas

Ruta	Pantalla	Qué hace
/simulacion/parametros	Parámetros de la Corrida	Sube y valida el archivo de pedidos; define modo (5D o Colapso), algoritmo, fecha de inicio, umbrales de semáforo y flota; botón Iniciar Corrida. Avisa si hay corridas en curso.
/simulacion/mapa	Mapa en Vivo (Simulación)	Muestra la corrida 5D o Colapso. Al llegar FINALIZADO o COLAPSADO congela el mapa y abre el modal de resumen.
/operacion-diaria	Operación en Tiempo Real	Mapa del escenario DIARIO (nunca termina). Pestaña Registrar pedido (X, Y, cliente, cantidad, plazo).
Pedidos, Dashboard	Placeholders	Fuera de este corte.

3.2 Componentes principales

- **MapaEnVivo** (canvas): grilla de 70 x 50, almacenes, vehículos por tipo con anillo de semáforo, pedidos, ruta del vehículo seleccionado. Interpola el movimiento entre dos snapshots; zoom y desplazamiento son locales a cada navegador. Lo reutilizan Simulación y Operación Diaria.
- **Panel de unidad**: al hacer clic en un vehículo, muestra id, tipo, ETA, destino, carga y pedidos asignados.
- **Panel lateral**: leyenda y pestañas Vehículos y Pedidos con buscador; en Operación Diaria, también Registrar pedido.
- **Modal de resumen**: pedidos totales, entregados, atrasados, no asignados y costo total; solo en 5D y Colapso.

3.3 Capa de datos

El mapa no sabe de dónde vienen los datos: consume una interfaz SnapshotSource con start(onSnapshot) y stop(). Hay dos implementaciones y una función createSource decide cuál usar.

Pieza	Función
MockSource	Simula una corrida creíble (37 vehículos, pedidos, semáforo, cierre tras N ticks). Permite desarrollar y hacer la demo sin backend.
LiveSource	Cliente STOMP: se suscribe a /topic/sim/{runId}, parsea cada mensaje como Snapshot y reconecta solo (3 s). Expone el estado de conexión (conectando, conectado, desconectado).
api.ts	Llamadas REST tipadas: getMapaBase, subirArchivoPedidos, iniciarEscenario, getEscenariosActivos, registrarPedido, getEstado. Errores como ApiError con status, código y mensaje.
Estado global	Context con parámetros de la corrida, archivo de pedidos y su validación, y un runId por escenario.

3.4 Variables de entorno

Variable	Ejemplo	Efecto
VITE_USE MOCK	true / false	true: sin llamadas de red (MockSource y respuestas REST simuladas). false: usa el backend real.
VITE_API_URL	http://localhost:8080	Base de los endpoints REST. Vacía si front y back se sirven desde el mismo origen.
VITE_WS_URL	ws://localhost:8080/ws	Endpoint WebSocket. En producción, wss://.
VITE MOCK FIN TICK S	90	Solo mock: ticks hasta el snapshot final.

Para pasar del mock al backend real basta con cambiar VITE_USE MOCK a false y apuntar las dos URLs al servidor.

4. Backend (Planificador)

Aplicación Spring Boot 3 (Java 17) que contiene el dominio, la ingesta, los algoritmos, el motor de simulación y las interfaces REST y WebSocket. Los paquetes separan lo que no debe mezclarse: el algoritmo solo conoce dominio y planificador; nada de web.

4.1 Estructura de paquetes

com.paqrap/	
config/	WebSocketConfig, CorsConfig, ScenarioDefaults (grilla, flota, almacenes)
web/	
rest/	MapaController, ArchivoController, EscenarioController, PedidoController
dto/	SnapshotDto, VehiculoDto, PedidoDto, AlmacenDto, RunInfo, ParametrosCorrida
ws/	SnapshotPublisher
ingesta/	CsvRecordParser, OrderRecordParser, ScenarioDataLoader, ArchivoPedidosStore
dominio/	Order, Vehicle, VehicleType, Route, Warehouse, GridMap, SimTime, Solution
planificador/	PlanningAlgorithm, AlgorithmSelector, FeasibilityValidator, CostFunction, AStarPathfinder, greedy/GreedyAlgorithm, alns/ALNSAlgorithm, ats/ATSAAlgorithm
simulacion/	ScenarioRunner, Corrida, SimulationEngine, SimulationClock, ScenarioConfig, SnapshotMapper, StopCondition

4.2 Clases clave

Clase	Responsabilidad
ScenarioRunner	Servicio que guarda un Map de tipo de escenario a Corrida. Crea, consulta y detiene corridas. Rechaza con 409 una segunda corrida del mismo tipo.
Corrida	Agrupar el estado de una corrida: ScenarioConfig, reloj, motor, topic, estado (EN_CURSO, FINALIZADO, COLAPSADO, DETENIDA) y su ScheduledExecutor.
SimulationClock	Único lugar donde se conoce la hora. Tiempo real (factor 1) o simulado acelerado. El dominio nunca llama a la hora del sistema.
SimulationEngine	En cada tick: avanza el reloj, ingresa pedidos, mueve vehículos, dispara la planificación, actualiza semáforos, evalúa la parada y publica.
PlanningAlgorithm	Interfaz Strategy: Solution plan(ProblemInstance, Duration). Implementaciones: GreedyAlgorithm (respaldo), ALNSAlgorithm, ATSAAlgorithm (stub).
FeasibilityValidator, CostFunction	Compartidos por todos los algoritmos: capacidad, stock, plazo, jornada; costo $C = w_1 \text{ transporte} + w_2 \text{ SLA} + w_3 \text{ no asignados}$.
AStarPathfinder	Ruteo ortogonal sobre la grilla con caché de distancias entre pares de nodos.
ScenarioDataLoader, OrderRecordParser	Ingesta del archivo de pedidos y construcción de la ProblemInstance. Responsabilidad del bloque de ingesta.
SnapshotMapper	Convierte el estado de dominio al SnapshotDto del contrato, para que el dominio no dependa del JSON.
SnapshotPublisher	Envía el snapshot a /topic/sim/{runId} con SimpMessagingTemplate.

4.3 Los tres escenarios sobre el mismo motor

	DIARIO (Operación Diaria)	5D (Simulación)	COLAPSO
Fuente de pedidos	POST /api/pedidos en vivo (archivo opcional)	Archivo subido en el wizard	Archivo subido en el wizard
Reloj	Tiempo real, factor 1x	Acelerado: 5 días en 30-60 min (aprox. 160x)	Acelerado, igual o mayor que 5D
Condición de parada	Ninguna: nunca termina	Fin del día 5 (120 h simuladas)	Primer pedido que incumple su plazo o queda no asignable (a confirmar con el profesor)
Estado final	No aplica	FINALIZADO + resumen	COLAPSADO + causa + resumen

Los pedidos de 5D y Colapso entran al motor cuando el reloj simulado alcanza su hora de registro. El front no envía ningún factor de velocidad: lo fija el backend por escenario (RF-068).

4.4 Ciclo del motor

Cada corrida tiene su propio ScheduledExecutor que ejecuta un tick por segundo real. El algoritmo nunca corre dentro del tick: se lanza en un hilo aparte para que el mapa siga fluyendo mientras piensa.

```

cada tick (1 s real), en el hilo de la corrida:
  clock.avanzar() // dt simulado segun el factor del escenario
  ingresarPedidos(hasta = clock.now()) // del archivo (5D, Colapso) o los llegados por POST (DIARIO)
  moverVehiculos(dt) // avanza por su ruta segun la velocidad del tipo
  si toca planificar y no hay planificacion en curso:
    instancia = construirProblemInstance() // copia inmutable del estado actual
    planificador.submit(() -> algoritmo.plan(instancia, timeLimit)) // hilo aparte
  si hay Solution lista: aplicarSolution() // asigna rutas a vehiculos libres o por salir
  actualizarSemaforos(); evaluarParada()
  publisher.publicar(SnapshotMapper.map(estado)) // snapshot COMPLETO, no incremental

```

4.5 Integración del algoritmo

- **Contrato:** Solution plan(ProblemInstance instance, Duration timeLimit). La entrada trae pedidos pendientes, flota con posición y carga, almacenes con stock, hora simulada y grilla. La salida trae, por vehículo, la lista ordenada de paradas con cantidad y ETA.
- **Por lotes:** se planifica cada 30-60 minutos simulados (unos 11-22 s reales a 160x) o ante un pedido urgente, con un límite de 1 a 3 segundos reales. En DIARIO se planifica con un intervalo propio en tiempo real.
- **Qué se replanifica:** solo pedidos pendientes o planificados que aún no salieron. Lo que ya va en ruta no se modifica (RF-019).
- **Concurrencia:** el algoritmo trabaja sobre una copia inmutable y el resultado lo aplica el hilo del tick, así no hay estado compartido modificable.
- **Respaldo:** GreedyAlgorithm (pedido más urgente primero, al vehículo factible más cercano) garantiza que los 3 escenarios funcionen aunque el ALNS no esté listo. AlgorithmSelector elige la clase según ScenarioConfig.algoritmo.
- **Movimiento:** los vehículos avanzan por rutas ortogonales según su velocidad (km/h) con 1 km por celda; la posición puede ser fraccionaria dentro de un tramo, y el front la interpola.

4.6 Ingesta del archivo de pedidos

El front sube el archivo por REST (multipart). El backend lo parsea con OrderRecordParser, que es la validación oficial; la del front solo da respuesta inmediata.

Elemento	Detalle
Formato de línea	DDdHHhMMm:posX,posY,idCliente,cantidad,horizonteHoras (ejemplo: 01d00h00m:31,14,c9103,04,04)
Validaciones	Formato de la línea; cantidad mayor que 0; horizonte en {4, 8, 12, 18, 36}; coordenadas dentro de la grilla (X de 0 a 70, Y de 0 a 50).
Resultado	Order con deadline = registeredAt + horizonte. Los pedidos válidos se guardan en ArchivoPedidosStore (memoria) bajo un archivold.
Respuesta del endpoint	{ archivold, totalPedidos, errores: [{ linea, mensaje }] }. Si hay errores, el front no deja iniciar la corrida.
Uso posterior	iniciar recibe el archivold y el motor toma los pedidos del almacén en memoria.

4.7 Datos de configuración (ScenarioDefaults)

Vehículo	Cantidad	Capacidad (paq.)	Velocidad	Costo por km
Auto	10	24	40 km/h	S/ 8.00
Moto	15	8	25 km/h	S/ 6.00
Bicicleta	12	4	12 km/h	S/ 3.00

Almacén	Id	Posición (X, Y)	Capacidad	Recarga
Central	CENTRAL	(27, 14)	Infinita	Permanente
Intermedio Nor-Oeste	INTERMEDIO_NOROESTE	(12, 38)	1 000 unidades	Diaria 23:59:59
Intermedio Este	INTERMEDIO_ESTE	(57, 27)	1 000 unidades	Diaria 23:59:59

Grilla de 70 x 50 nodos, 1 km entre nodos adyacentes. Semáforo por holgura de plazo: verde con 60% o más, ámbar de 30% a 60%, rojo con menos de 30% (configurable desde el wizard). Estos valores viven en un solo lugar del back y en defaults.ts del front.

5. Comunicación Front - Back

5.1 Criterio de uso

Canal	Cuándo	Quién inicia	Ejemplos
REST (HTTP)	Una vez, cuando el usuario hace algo	El navegador	Subir archivo, iniciar escenario, registrar pedido, pedir la base del mapa
WebSocket (STOMP)	Continuo, cada tick	El servidor empuja	Snapshot de vehículos, pedidos y almacenes; evento final

Dentro del backend, Planificador y publicador no usan REST entre sí: el motor llama al algoritmo como una llamada Java normal y entrega el snapshot al publicador. REST existe solo en la frontera con el navegador.

5.2 Endpoints REST

Método	Ruta	Uso	Respuesta
GET	/api/mapa/base	Grilla, almacenes y umbrales de semáforo	200 con el JSON de la sección 6.2
POST	/api/archivos/pedidos	Subir el archivo de pedidos (multipart, campo file)	200 { archivold, totalPedidos, errores[] }
POST	/api/escenarios/{tipo}/iniciar	Crear la corrida. tipo = DIARIO, 5D o COLAPSO	200 { runId, escenario, estado, topic }; 409 si ya hay una de ese tipo
GET	/api/escenarios/activos	Corridas en curso (para dispositivos que entran tarde)	200 con una lista, posiblemente vacía
GET	/api/escenarios/{runId}/estado	Último snapshot (respaldo si falla el WebSocket)	200 Snapshot; 404
POST	/api/pedidos	Registrar un pedido en la corrida DIARIO	201 con el pedido y su deadline; 409 si DIARIO no está activa
GET	/api/pedidos/{id}	Detalle de un pedido	200; 404

Errores: cuerpo JSON uniforme { "error": "CODIGO", "mensaje": "..." } con 400 (validación), 404 y 409. No existe ningún endpoint de velocidad (RF-068).

5.3 WebSocket con STOMP

STOMP es un protocolo de mensajería simple (publicar y suscribirse) que se transporta sobre WebSocket. Spring Boot lo trae integrado con un broker en memoria, suficiente para este sistema. La comunicación es de una sola vía: el servidor publica, el navegador solo escucha.

Elemento	Valor
Endpoint de conexión	/ws (WebSocket nativo; no hace falta SockJS)
Topic por corrida	/topic/sim/{runId}: cada corrida publica en el suyo, así cada dispositivo ve el escenario que elija
Mensaje	JSON del Snapshot (sección 6.1). Es el estado completo, no un delta

Elemento	Valor
Frecuencia	Un snapshot por tick (1 por segundo)
Reconexión	El cliente reintenta cada 3 s. Como cada snapshot es completo, al reconectar el siguiente mensaje deja el mapa consistente, sin sincronización extra
Cierre de corrida	Último mensaje con estado FINALIZADO o COLAPSADO y el resumen
Producción	wss:// y permitir el origen del front tanto en REST como en ws

Configuración en el servidor (referencia):

```
@Configuration @EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
    public void registerStompEndpoints(StompEndpointRegistry r) {
        r.addEndpoint("/ws").setAllowedOriginPatterns(allowedOrigins); // desde variable de entorno
    }
    public void configureMessageBroker(MessageBrokerRegistry r) {
        r.enableSimpleBroker("/topic"); // broker en memoria
    }
}
// SnapshotPublisher
messagingTemplate.convertAndSend("/topic/sim/" + runId, snapshotDto);
```

Cliente en el navegador (referencia):

```
const client = new Client({ brokerURL: import.meta.env.VITE_WS_URL, reconnectDelay: 3000 });
client.onConnect = () => client.subscribe(`/topic/sim/${runId}`,
    msg => onSnapshot(JSON.parse(msg.body) as Snapshot));
client.activate();
```

5.4 Secuencia de una corrida

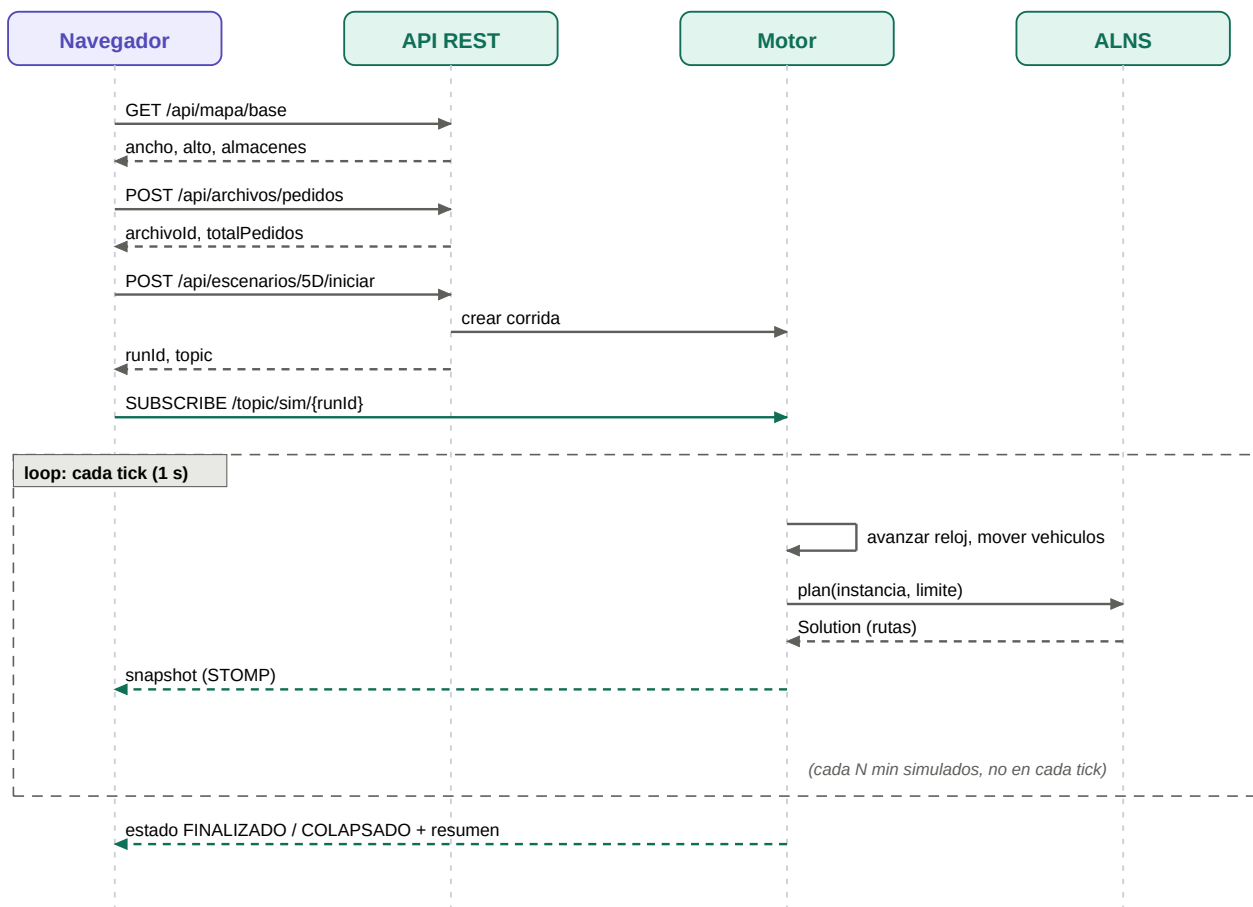


Figura 2. Secuencia: base del mapa, subida de pedidos, inicio, suscripción y ciclo de snapshots hasta el cierre.

5.5 Multi-dispositivo

- La corrida vive en el servidor; cada dispositivo solo se suscribe a su topic. Zoom y zona visible son estado local de cada navegador, así que no se afectan entre sí.

- Un dispositivo que entra en cualquier momento llama a GET /api/escenarios/activos, elige el escenario (DIARIO, 5D o COLAPSO) y se suscribe a su topic. Desde el siguiente tick ve el mapa.
- Como coexisten corridas de tipos distintos, el front muestra en Parámetros y en el menú lateral cuáles están en curso.

6. Contrato JSON

Los enums son parte del contrato y no deben cambiar sin avisar a ambos lados: TipoVehiculo (AUTO, MOTO, BICICLETA), EstadoVehiculo (EN_RUTA, REGRESANDO, ATENDIENDO, LIBRE), EstadoPedido (PENDIENTE, PLANIFICADO, EN_RUTA, ENTREGADO), Semaforo (VERDE, AMBAR, ROJO), EstadoCorrida (EN_CURSO, FINALIZADO, COLAPSADO) e IdAlmacen (CENTRAL, INTERMEDIO_NOROESTE, INTERMEDIO_ESTE). simTime usa el formato DDdHHhMMm; en DIARIO, simTimeIso trae la hora real.

6.1 Snapshot (WebSocket, cada tick)

```
{
  "runId": "run-5d-001", "escenario": "5D", "estado": "EN_CURSO",
  "simTime": "02d14h30m", "simTimeIso": "2026-01-02T14:30:00",
  "vehiculos": [{
    "id": "A-01", "tipo": "AUTO", "x": 31, "y": 14, "estado": "EN_RUTA",
    "almacenBase": "CENTRAL", "carga": 18, "capacidad": 24,
    "ruta": [[31,14],[31,15],[32,15],[34,15]],
    "proximaParada": { "pedidoId": "P-000123", "x": 34, "y": 15, "eta": "02d15h10m" },
    "semaforo": "VERDE" }],
  "pedidos": [{
    "id": "P-000123", "clienteId": "c9103", "x": 34, "y": 15, "cantidad": 4,
    "registradoEn": "02d12h00m", "deadline": "02d16h00m",
    "estado": "EN_RUTA", "vehiculoId": "A-01", "semaforo": "VERDE" }],
  "almacenes": [
    { "id": "CENTRAL", "x": 27, "y": 14, "stock": null },
    { "id": "INTERMEDIO_NOROESTE", "x": 12, "y": 38, "stock": 740 },
    { "id": "INTERMEDIO_ESTE", "x": 57, "y": 27, "stock": 905 }],
  "metricas": { "pedidosTotal": 412, "entregados": 360, "enRuta": 38, "pendientes": 14, "atrasados": 0 }
}
```

stock null significa inventario infinito (almacén central). El snapshot incluye solo los pedidos activos para mantener el mensaje liviano.

6.2 Base del mapa (GET /api/mapa/base)

```
{ "ancho": 70, "alto": 50,
  "almacenes": [
    { "id": "CENTRAL", "nombre": "Almacén Central", "x": 27, "y": 14, "tipo": "CENTRAL" },
    { "id": "INTERMEDIO_NOROESTE", "nombre": "Intermedio Nor-Oeste", "x": 12, "y": 38, "tipo": "INTERMEDIO", "capacidad": 1000 },
    { "id": "INTERMEDIO_ESTE", "nombre": "Intermedio Este", "x": 57, "y": 27, "tipo": "INTERMEDIO", "capacidad": 1000 }],
  "semaforo": { "verdeMinPct": 60, "ambarMinPct": 30 } }
```

6.3 Iniciar una corrida (POST /api/escenarios/{tipo}/iniciar)

```
// request (archivoPedidosId es opcional en DIARIO)
{ "algoritmo": "ALNS", "fechaInicio": "2026-01-01T00:00:00", "archivoPedidosId": "arch-17",
  "flota": { "auto": 10, "moto": 15, "bicicleta": 12 },
  "umbralesSemaforo": { "verdeMinPct": 60, "ambarMinPct": 30 } }
// response
{ "runId": "run-5d-001", "escenario": "5D", "estado": "EN_CURSO", "topic": "/topic/sim/run-5d-001" }
```

6.4 Registrar pedido (POST /api/pedidos) y evento final

```
// request
{ "x": 34, "y": 15, "clienteId": "c9103", "cantidad": 4, "horizonteHoras": 12 }
// 201
{ "id": "P-000150", "clienteId": "c9103", "x": 34, "y": 15, "cantidad": 4,
  "registradoEn": "02d14h20m", "deadline": "03d02h20m", "estado": "PENDIENTE" }

// ultimo mensaje del topic en Colapso (en 5D: estado FINALIZADO, sin causa)
{ "runId": "run-col-001", "escenario": "COLAPSO", "estado": "COLAPSADO", "simTime": "09d03h45m",
  "causa": "PEDIDO_INCUMPLIO_PLAZO",
  "resumen": { "pedidosTotal": 3120, "entregados": 2790, "atrasados": 41, "noAsignados": 12, "costoTotal": 18450.50 } }
```

7. Despliegue

La solución debe estar integrada y desplegada, accesible desde cualquier dispositivo por la web.

Opción	Cómo	Consideraciones
A (recomendada): un solo servicio	Se compila el front y se incluye en static/ del JAR. Un Dockerfile multi-stage genera la imagen. Una sola URL, sin CORS.	Menos puntos de falla. VITE_API_URL y VITE_WS_URL pueden ir vacías o relativas al mismo origen.

Opción	Cómo	Consideraciones
B: front y back separados	Back en Docker (Render, Railway o Fly.io); front en Vercel o similar.	Requiere CORS para REST y /ws, y wss://. Más piezas que coordinar.

- El puerto se toma de la variable PORT y los orígenes permitidos de una variable de entorno.
- Verificar que el hosting permita WebSocket y que el plan elegido no duerma el servicio antes de la presentación.
- Las corridas viven en memoria: reiniciar el servicio las pierde. Conviene que DIARIO arranque automáticamente al levantar la aplicación, de modo que cualquier dispositivo la encuentre siempre encendida.

8. Riesgos y pendientes por confirmar

Tema	Situación	Acción
Tamaño de la grilla	El informe de algoritmos muestra 71 x 51 en unas secciones y 70 x 50 en otra. Este documento y las pantallas usan 70 x 50.	Mantenerlo como una sola constante y confirmar con el equipo.
Colapso: definición	Se asume parada ante el primer pedido que incumple su plazo o no puede asignarse.	Confirmar el criterio con el profesor.
DIARIO a tiempo real	A factor 1x un vehículo tarda mucho en moverse de forma visible (una bicicleta recorre 1 km en 5 min).	Aclarar cómo se presentará este escenario y, si hace falta, acordar un factor moderado.
Carga de CPU	Tres corridas con ALNS a la vez en un hosting modesto.	Límite de tiempo corto por planificación, intervalos más largos en DIARIO, y pruebas con las tres activas.
WebSocket en producción	Algunos hostings gratuitos limitan o duermen conexiones.	Probar el despliegue temprano. Plan B: consultar GET /estado cada 2-3 s.
Tamaño del snapshot	37 vehículos más pedidos activos, un mensaje por segundo.	Enviar solo pedidos activos y rutas restantes; medir antes de optimizar.
Evaluación individual	Se evalúa la programación de cada estudiante.	Cada integrante sube su parte con su propio usuario de Git.

9. Plan de implementación del backend

Orden pensado para que el front pueda conectarse al backend real lo antes posible y para que, si el tiempo se acaba, lo imprescindible ya funcione.

Paso	Qué se construye	Bloque	Hito
1	Esqueleto Spring Boot: DTOs del contrato, WebSocketConfig, SnapshotPublisher, endpoints de base y de escenarios con datos de prueba	Visualizador / Planificador	El front se conecta al back real (VITE_USE MOCK=false)
2	Ingesta: OrderRecordParser, ScenarioDataLoader, ArchivoPedidosStore y POST /api/archivos/pedidos	Ingesta	El wizard sube y valida pedidos de verdad
3	Dominio, SimulationClock y SimulationEngine con movimiento ortogonal y GreedyAlgorithm	Planificador	Vehículos se mueven y entregan en el mapa
4	AStarPathfinder con caché y ScenarioRunner con una corrida por tipo	Planificador	Coexisten DIARIO, 5D y COLAPSO
5	Integración del ALNS por Strategy con FeasibilityValidator y CostFunction	Planificador	ALNS planifica por lotes
6	Condiciones de parada (5D, Colapso), POST /api/pedidos y arranque automático de DIARIO	Planificador / Ingesta	Los 3 escenarios completos
7	Dockerfile, despliegue y pruebas de punta a punta con varios dispositivos	Todos	Solución integrada y desplegada

Prioridad si falta tiempo: primero 5D corriendo y visible desde otro dispositivo; luego Colapso; luego Operación Diaria con registro manual; ALNS real puede reemplazar al algoritmo goloso al final porque ambos implementan la misma interfaz.